

What is Virtual Memory?

||Demand Paging|| Page Faults

1.What is Virtual Memory?

Virtual Memory is a technique that allows a computer to run programs **even when the entire program is not present in main memory (RAM)**.

The operating system **creates an illusion** that the system has a **very large main memory**, even though the actual RAM is limited.

It does this by **using part of the hard disk (secondary memory)** as an extension of RAM. This disk area is called **swap space**.

Real-World Example

Imagine:

- Your **study table** = RAM (small but fast)
- Your **bookshelf** = Hard disk (large but slow)

You **don't keep all books on the table at once**.

You keep **only the books you are currently studying**.

When you need another book, you **bring it from the shelf**.

This is **Virtual Memory**.

2.Why Do We Need Virtual Memory?

Earlier, a program **had to fully fit into RAM** to execute.

This caused problems:

- Large programs **could not run**
- RAM got **exhausted quickly**
- CPU stayed idle due to fewer processes

But in reality:

A program does not need all its parts at the same time

So, virtual memory allows **partial loading** of programs.

3.Advantages of Virtual Memory

1.Programs can be larger than physical memory

Even if RAM is 4 GB, you can run programs needing 10 GB.

2. More programs can run simultaneously

Since each program uses less RAM at a time,

CPU utilization increases

System throughput improves

3. Better experience for user and system

- Faster response
- Efficient memory usage
- No memory limitation anxiety

Programmer's View

From the programmer's perspective:

- Memory looks **huge and continuous**
- They don't worry about actual RAM size

OS handles everything behind the scenes.

4. What is Demand Paging?

Demand Paging is the **most popular virtual memory technique**.

Idea:

“Load a page **only when it is needed**.”

Real-World Example

Suppose you are preparing for an exam:

- You don't read the **entire book at once**
- You read **only the chapter currently required**

Similarly:

- OS loads **only required pages**
- Unused pages stay on disk

5. How Demand Paging Works

Step-by-Step:

1. Program starts execution

2. OS **does NOT load entire program** into RAM

3. Only the **needed pages** are loaded

What is a Page Fault?

When a program tries to access a page:

- If page is **not in RAM**
- → **Page Fault occurs**

OS:

- Pauses the process
- Brings the required page from disk to RAM
- Resumes execution

This happens **automatically**.

6. Page Replacement Algorithms

RAM is limited, so:

- If RAM is full
- OS must **remove some page**

Page Replacement Algorithms decide:

- Which page to remove
- Examples: FIFO, LRU, Optimal (you'll study these later)

7. Lazy Swapper (Very Important Concept)

Earlier:

- Entire process was swapped in and out

Now:

- OS uses a **Lazy Swapper**

Meaning:

“Never bring a page into memory unless it is actually needed.”

This saves:

- Time
- RAM
- Disk operations

8. Swapper vs Pager (Clarification)

Term	Meaning
Swapper	Moves entire processes
Pager	Moves individual pages

Since virtual memory works with **pages**,
The correct term is **Pager**, not Swapper.

9.Valid-Invalid Bit Scheme

Each page table entry has a **Valid-Invalid bit**.

Purpose:

To check whether a page:

- Is in RAM
- Or on disk
- Or illegal

Meaning of Bits

Valid bit = 1

- Page is **legal**
- Page is **in main memory**

Invalid bit = 0

- Page is:
 - Either **not part of the program**
 - Or **currently stored on disk**

Real-World Example

Think of a **library register**:

- ✓ = Book is available on desk
- ✗ = Book is either:
 - In storage
 - Or does not exist